МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

# ЛАБОРАТОРНА РОБОТА №3

з курсу: «Хмарні технології та сервіси»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2025

## Хід роботи

### 1. Додавання реплікації для підвищення надійності (durability)

**In this lab we expand on our previous lab by adding durability to our shards with replication. Each shard should have 3+ replicas. It is suggested to implement leader-based replication for simplicity, though brave enough can experiment with leaderless cluster.**

Розширити попередню лабу шляхом додавання реплікації шардів з метою підвищення надійності зберігання даних. Забезпечити наявність не менше ніж трьох реплік для кожного логічного шарду (replication factor ≥ 3). Обрати та реалізувати модель реплікації (leader-based) replication, як простіший підхід у реалізації.

shard1:
leader
follower1
follower2

shard2:
leader
follower1
follower2

```
shard1-leader:
  build: .
  ports:
    - "8001:8000"
  environment:
```

```
shard2-leader:
  build: .
  ports:
    - "8004:8000"
  environment:
```

```
shard1-follower1:
  build: .
  ports:
    - "8002:8000"
  environment:
```

```
shard2-follower1:
  build: .
  ports:
    - "8005:8000"
  environment:
```

```
shard1-follower2:
  build: .
  ports:
    - "8003:8000"
  environment:
```

```
shard2-follower2:
  build: .
  ports:
    - "8006:8000"
  environment:
```

### 2. Реалізація replica set та обробка операцій запису

**Note that conflict resolution is for the next lab, here we keep things simple. Implement replica set per each shard. Leader node receives writes and event-sources (fan-out) them with some log structure. Think about delivery order and versions.**

У другому пункті реалізувємо replica set для кожного shard-а. Координатор тепер знає leader і follower-и, а shard-и реєструються з ролями. Тобто по суті метою даного пункту є об'єднання shard-вузлів у replica set та введення ролей leader і follower. Leader-вузол надалі відповідатиме за обробку операцій запису, тоді як follower-вузли використовуватимуться для забезпечення надійності та масштабування читання.

Тут ми додаємо структуру shard_replicas, яка використовується координатором для зберігання інформації про replica set кожного шарду. У цій структурі окремо зберігаються адреси leader-вузла та follower-вузлів.

При першій реєстрації нового shard-а створюється відповідний replica set, а логічний shard додається до consistent hash ring. Це дозволяє виконувати шардінг незалежно від кількості фізичних реплік.

```
if shard_id not in shard_replicas:
    shard_replicas[shard_id] = {
        "leader": None,
        "followers": []
    }
```

У shard_app.py реалізовано механізм реєстрації shard-вузла при старті контейнера. Кожен вузол передає координатору ідентифікатор шарду, власну адресу та роль. Таким чином координатор отримує повну інформацію про структуру replica set і може надалі коректно маршрутизувати запити.

```
SHARD_ID = os.getenv("SHARD_ID")
ROLE = os.getenv("ROLE")  # leader | follower
```

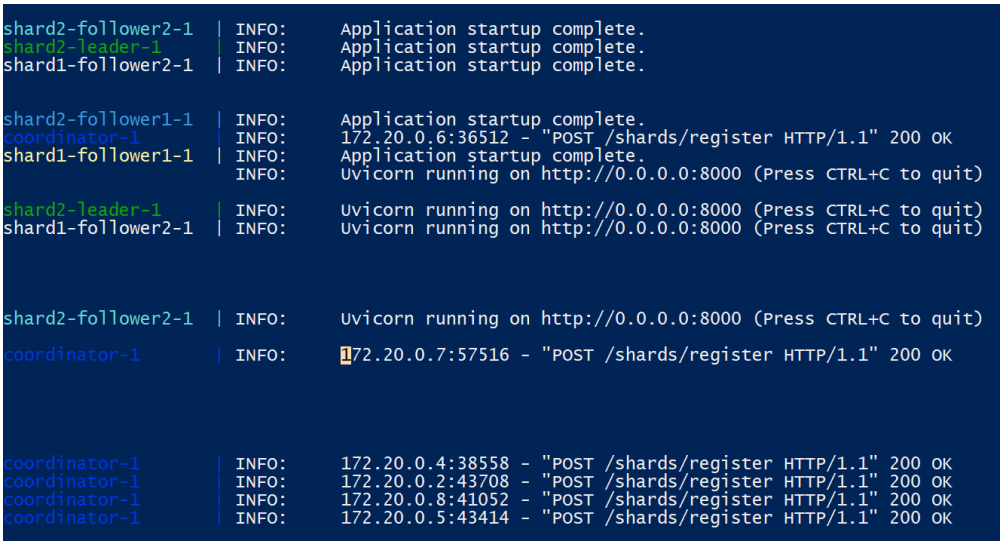
Coordinator тепер отримує повну інформацію про shard-вузол

```
payload = {
    "shard_id": SHARD_ID,
    "address": SHARD_ADDRESS,
    "role": ROLE,
}
```

Shard-вузол тепер: знає якому shard-у належить знає чи він leader, чи follower




Replica set реалізовано через запуск кількох shard-контейнерів для одного shard-а, їх реєстрацію у coordinator з ролями leader і follower, а hash ring працює з логічними shardами.




## 3. Реплікація даних та балансування навантаження на читання

**Read replicas (followers) consume log. Note that they should be used not only for durability, but also for load distribution – read load should be evenly balanced. So controller should resolve reads to all active shards.**

Далі необхідно реалізувати механізм реплікації на основі leader-based підходу. Leader вузол shard повинен приймати всі операції запису та виконувати event sourcing, після чого розсилати події (fan-out) на follower-репліки. Follower-вузли повинні споживати ці події та підтримувати актуальну копію даних

У реалізованій системі coordinator відповідає за маршрутизацію запитів. Для кожного ключа за допомогою consistent hashing визначається логічний shard. Далі coordinator використовує інформацію про replica set цього shard, щоб направляти операції запису на leader, а операції читання на leader або одну з follower-реплік

Усі операції запису (PUT) обробляються виключно leader-вузлом відповідного shard.

```
leader = shard_replicas[shard_id]["leader"]

 r = requests.get(f"{target}/tables/{table}/records/{key}")
 if r.status_code >= 400:
     raise HTTPException(r.status_code, r.text)
```

READ (GET)  leader або follower ( балансування навантаження на читання ), згідно умови завдання для операцій читання coordinator не обмежується лише leader-вузлом, а обирає одну з доступних реплік shard-а.

```
replicas = []
leader = shard_replicas[shard_id]["leader"]
if leader:
    replicas.append(leader)
replicas.extend(shard_replicas[shard_id]["followers"])

if not replicas:
    raise HTTPException( status_code: 500,  detail: "No replicas available")

target = random.choice(replicas)
```

Coordinator випадковим чином обирає одну з реплік shard-а для обробки read-запиту. Це дозволяє рівномірно розподіляти навантаження між leader та follower-вузлами + покращення пропускної здатності системи (при читанні)

Event sourcing та fan-out реплікація, leader після прийняття запису формує подію, що містить ключ, значення та версію, і розсилає цю подію на всі follower-репліки shard.

```
event = {
    "table": table,
    "key": key,
    "value": value,
    "version": version
}

for follower in FOLLOWERS:
    requests.post(f"{follower}/internal/replicate", json=event)
```

Кожна операція запису перетворюється на подію з чітким порядком. Leader гарантує порядок подій та виконує fan-out,  розсилку цієї події на всі follower-вузли, тобто це і є підхід event sourcing (стан системи відновлюється шляхом послідовного застосування подій)

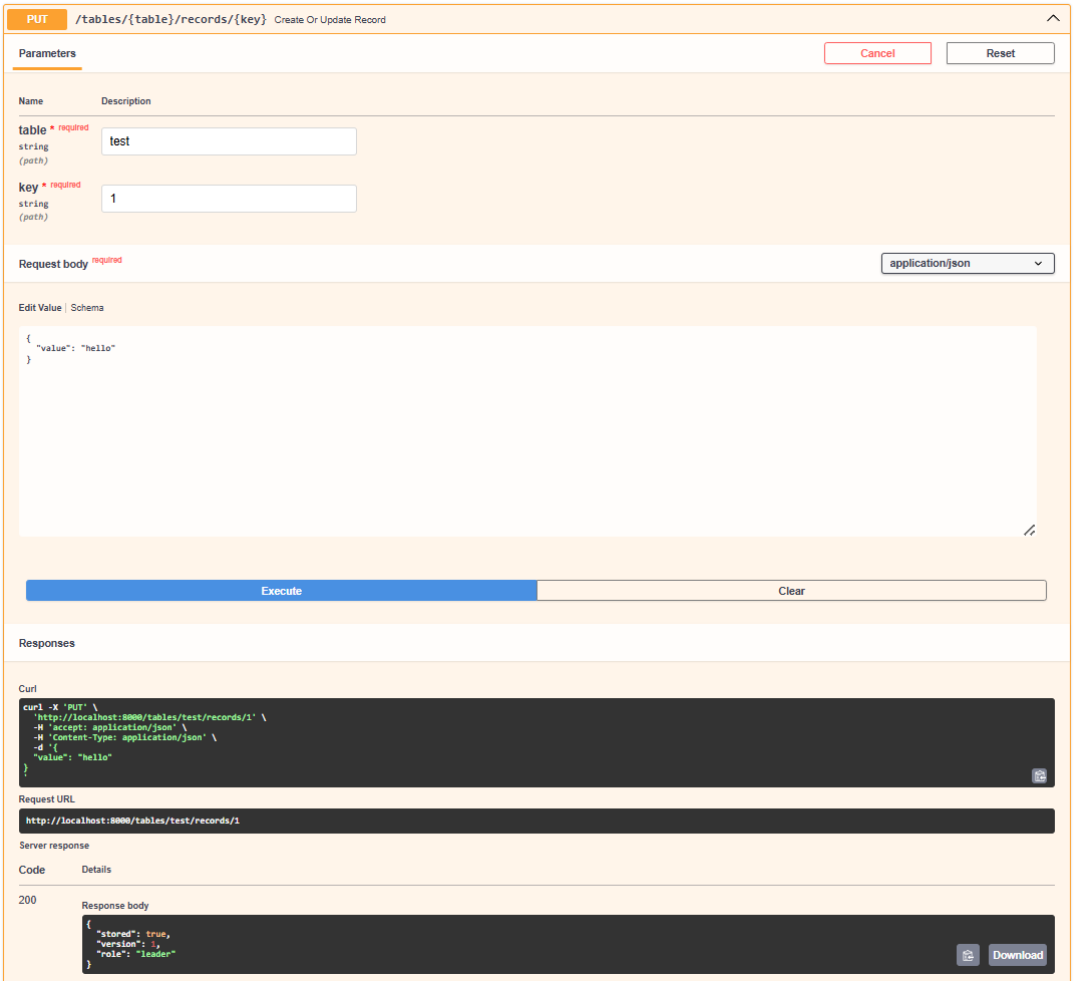
Споживання журналу подій follower-репліками це відбувоєтся завдяки Follower вузлам, які  не приймають записи напряму, а лише споживають події, які надсилає leader, і відтворюють стан локального сховища

```python
@app.post("/internal/replicate")
def replicate(event: dict):
    table = event["table"]
    key = event["key"]
    value = event["value"]

    storage.setdefault(table, {})[key] = value
    return {"status": "replicated"}
```

Follower отримує подію, застосовує її до локального сховища та таким чином підтримує актуальну копію даних. Це забезпечує durability, оскільки дані зберігаються на кількох вузлах.

Виконаємо WRITE :



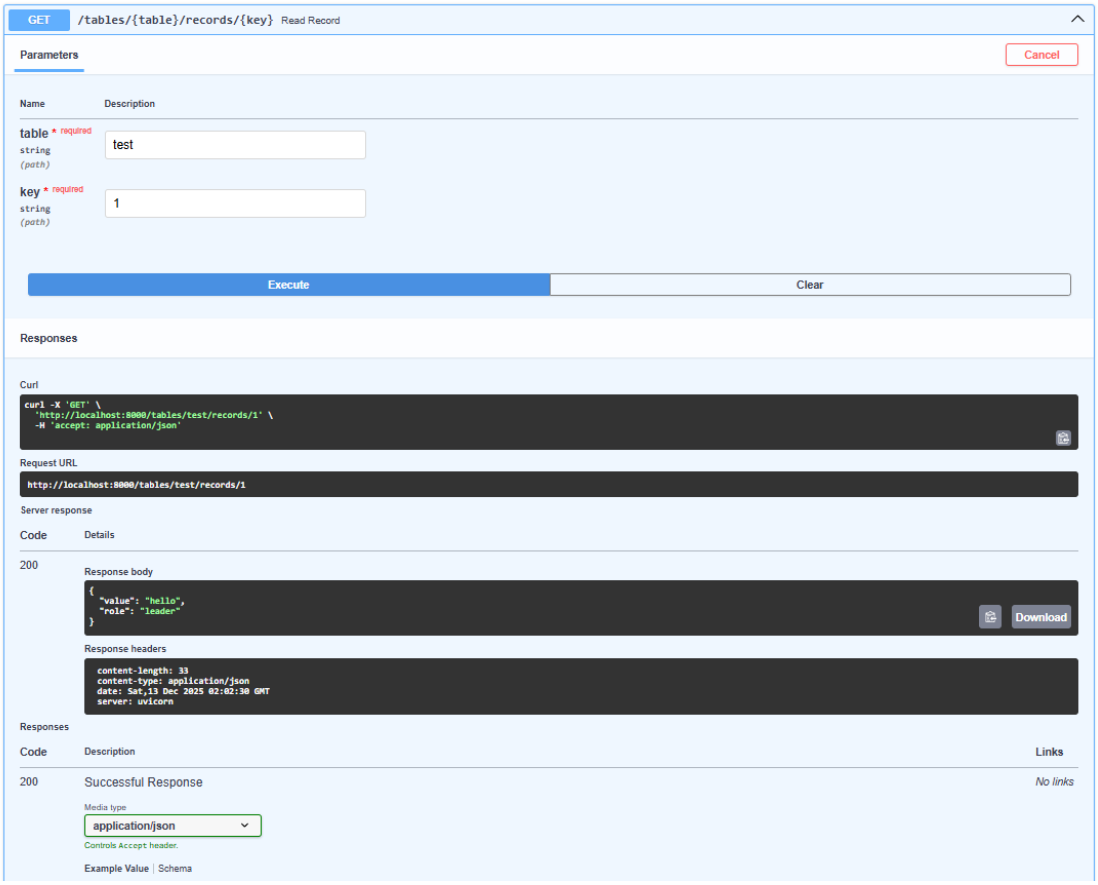

У логах маємо:

```
coordinator-1       | INFO:     172.20.0.1:52650 - "PUT /tables/test/records/1 HTTP/1.1" 200 OK
shard1-leader-1     | INFO:     172.20.0.5:46346 - "PUT /tables/test/records/1 HTTP/1.1" 200 OK
```

Leader прийняв запис і згенерував подію, яку розсилає follower-реплікам, тоді у логах для follower маємо

```
Shard shard1 registered as follower
INFO:     172.20.0.2:42888 - "POST /internal/replicate HTTP/1.1" 200 OK
```

Виконаємо GET




У логах маємо наступне (операцію я виконав +5 разів):

```
PS C:\Users\user\PycharmProjects\coordinator> docker logs coordinator-shard1-leader-1
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Shard shard1 registered as leader
INFO:     172.20.0.5:46346 - "PUT /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:55354 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:55366 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:56202 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:57718 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
PS C:\Users\user\PycharmProjects\coordinator>
```

```
PS C:\Users\user\PycharmProjects\coordinator> docker logs coordinator-shard1-follower1-1
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Shard shard1 registered as follower
INFO:     172.20.0.2:58818 - "POST /internal/replicate HTTP/1.1" 200 OK
INFO:     172.20.0.5:50042 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:50044 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
PS C:\Users\user\PycharmProjects\coordinator>
```

```
PS C:\Users\user\PycharmProjects\coordinator> docker logs coordinator-shard1-follower2-1
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Shard shard1 registered as follower
INFO:     172.20.0.2:42888 - "POST /internal/replicate HTTP/1.1" 200 OK
INFO:     172.20.0.5:58614 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:32996 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
INFO:     172.20.0.5:54856 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
PS C:\Users\user\PycharmProjects\coordinator>
```

Coordinator випадково обирає репліку shard-a для read-запиту, тому GET-запити обробляються різними вузлами.

Згідно завдання: Controller should resolve reads to all active shards, це можна побачити у логах:

```
shard1-follower2-1  | INFO:     172.20.0.5:58614 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
coordinator-1       | INFO:     172.20.0.1:32858 - "GET /tables/test/records/1 HTTP/1.1" 200 OK


coordinator-1       | INFO:     172.20.0.1:60878 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
shard1-follower2-1  | INFO:     172.20.0.5:32996 - "GET /tables/test/records/1 HTTP/1.1" 200 OK

shard1-follower1-1  | INFO:     172.20.0.5:50042 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
coordinator-1       | INFO:     172.20.0.1:43376 - "GET /tables/test/records/1 HTTP/1.1" 200 OK


shard1-follower1-1  | INFO:     172.20.0.5:50044 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
coordinator-1       | INFO:     172.20.0.1:43376 - "GET /tables/test/records/1 HTTP/1.1" 200 OK


coordinator-1       | INFO:     172.20.0.1:43376 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
shard1-follower2-1  | INFO:     172.20.0.5:54856 - "GET /tables/test/records/1 HTTP/1.1" 200 OK


shard1-leader-1     | INFO:     172.20.0.5:56202 - "GET /tables/test/records/1 HTTP/1.1" 200 OK




coordinator-1       | INFO:     172.20.0.1:47870 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
shard1-leader-1     | INFO:     172.20.0.5:57718 - "GET /tables/test/records/1 HTTP/1.1" 200 OK
```

Тобто Coordinator маршрутизує read-запити до всіх активних реплік shard-a ( у логах видно, що одну і ту саму операцію GET, яка завжди спочатку приходить у coordinator, після цього обробляється різними репліками одного shard: shard1-follower1, shard1-follower2 або shard1-leader

Тобто тут реалізовано повноцінну leader-based реплікацію. Leader-вузол приймає всі операції запису, формує події та виконує їх fan-out на follower-репліки. Follower-вузли споживають журнал подій, підтримують копію даних і використовуються для обробки операцій читання, що забезпечує балансування навантаження.

**4. Відстеження offset та синхронізація після перезапуску**

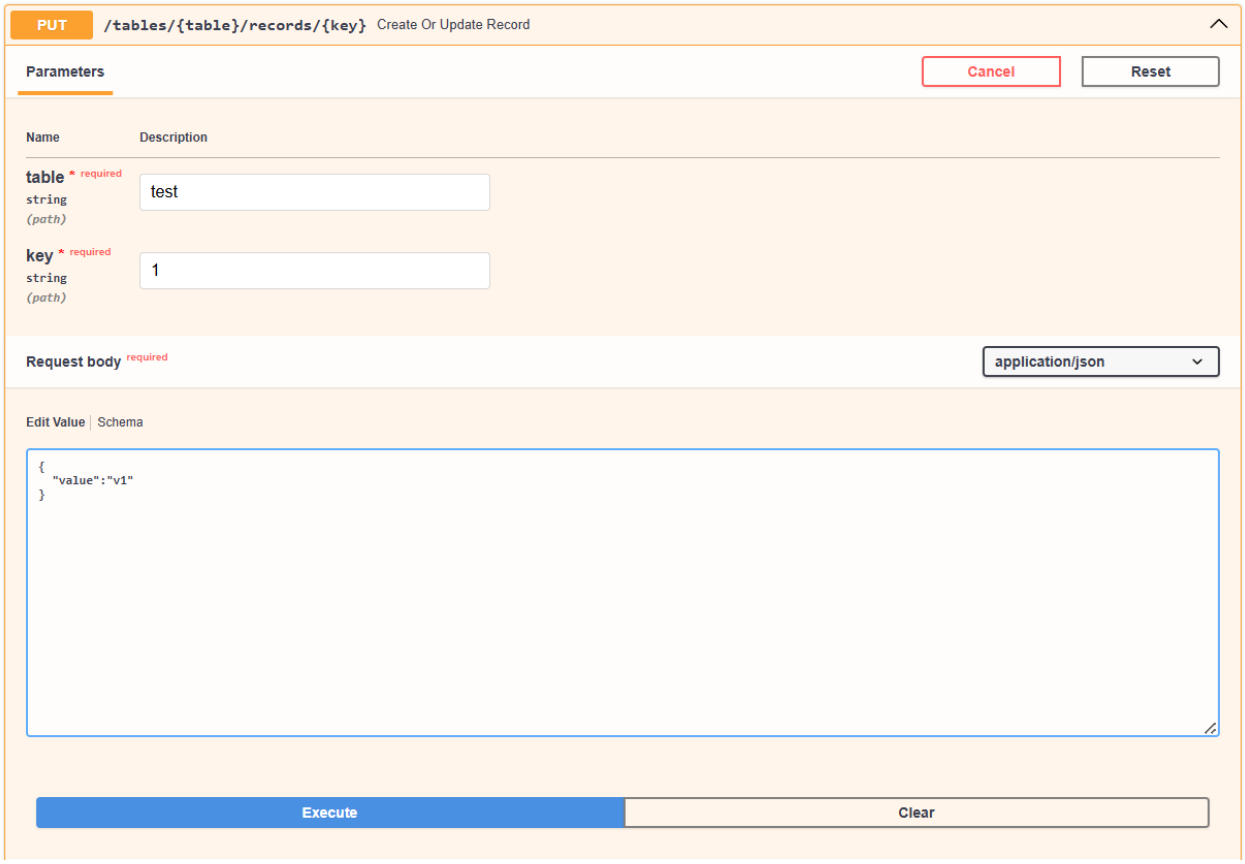
**Followers should track offset and upon restart resync only relevant log range.**

Тобто follower повинен пам’ятати, до якої події він уже синхронізувався, і після перезапуску не починати з нуля, а підтягнути лише ті події, яких йому бракує

Спочатку створюється таблиця через endpoint POST /tables з тілом запиту { "table_name": "test" }.




Далі зробимо три послідовні операції запису через PUT /tables/test/records/{key} (ключі : 1,2,3) з різними значеннями у форматі
value: v1
value: v2
value: v3

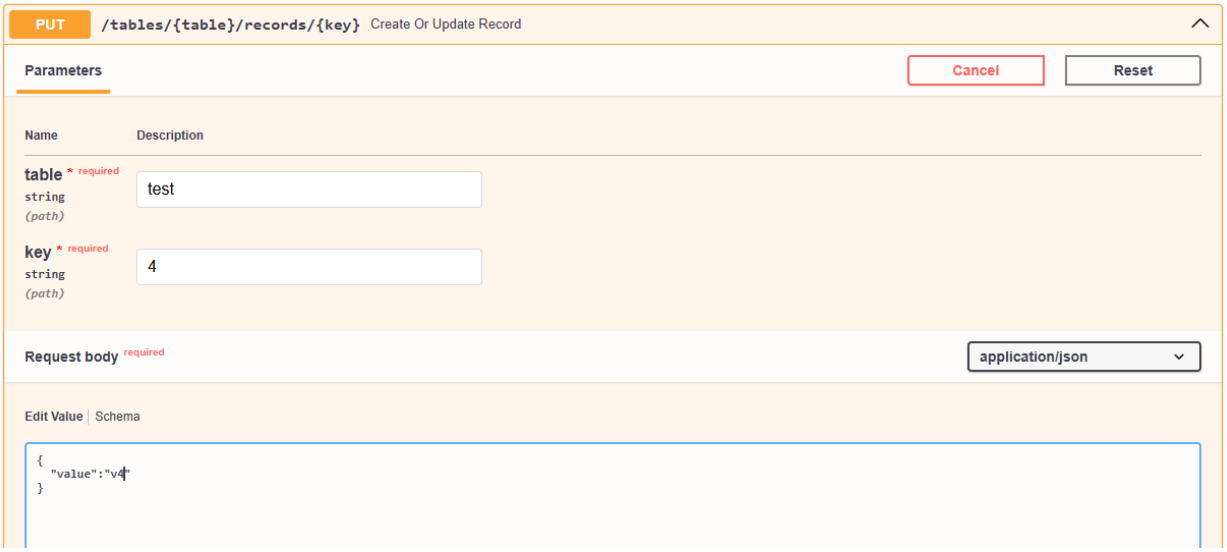



На цьому етапі follower-репліки отримують події через /internal/replicate та оновлюють свій offset.

Зупиняємо follower

```
PS C:\Users\user\PycharmProjects\coordinator> docker stop shard1-follower1
shard1-follower1
```

Додаємо ще 2 запити
value: v4
value: v5




Стартуємо follower

```
PS C:\Users\user\PycharmProjects\coordinator> docker start shard1-follower1
shard1-follower1
PS C:\Users\user\PycharmProjects\coordinator> docker logs shard1-follower1
```

а дивимося логи

```
PS C:\Users\user\PycharmProjects\coordinator> docker logs shard1-follower1
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
[follower] Loaded offset = 0
Resync failed: HTTPConnectionPool(host='shard1-leader', port=8000): Max ret
hard1-leader', port=8000): Failed to establish a new connection: [Errno 111
INFO:     172.20.0.7:47052 - "POST /internal/replicate HTTP/1.1" 200 OK
INFO:     172.20.0.7:51596 - "POST /internal/replicate HTTP/1.1" 200 OK
INFO:     172.20.0.7:41936 - "POST /internal/replicate HTTP/1.1" 200 OK
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [1]
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
[follower] Loaded offset = 3
[follower] Resynced up to version 5
```

З логів видно, що контейнер follower-а запустився FastAPI підняв сервер follower прочитав offset з volume ( На цьому етапі offset = 0, бо це перший старт з новим volume )

Resync failed: Connection refused бо follower одразу після startup намагається виконати Get запит, але leader ще не почав слухати порт

Далі приходять POST /internal/replicate (POST /internal/replicate HTTP/1.1 200 OK) leader продовжує працювати, coordinator маршрутизує PUT leader робить fan-out реплікацію a follower отримує події. Кожна така подія застосовується, offset збільшується та записується у файл.

Далі йде Shutting down (зупинка) після якої йде новий старт, а офсет стає
[follower] Loaded offset = 3
[follower] Resynced up to version 5

Тобто offset успішно зберігся (loaded offset = 3) follower пам’ятає, що він уже обробив 1 2 3 запити, leader повертає тільки події 4 і 5, follower застосовує їх та offset стає 5

## 5. Додаткові можливості та оцінка продуктивності (optional)

**(Optional) For reads implement quorum with R+W>N rule. Benchmark your solution with YCSB.**

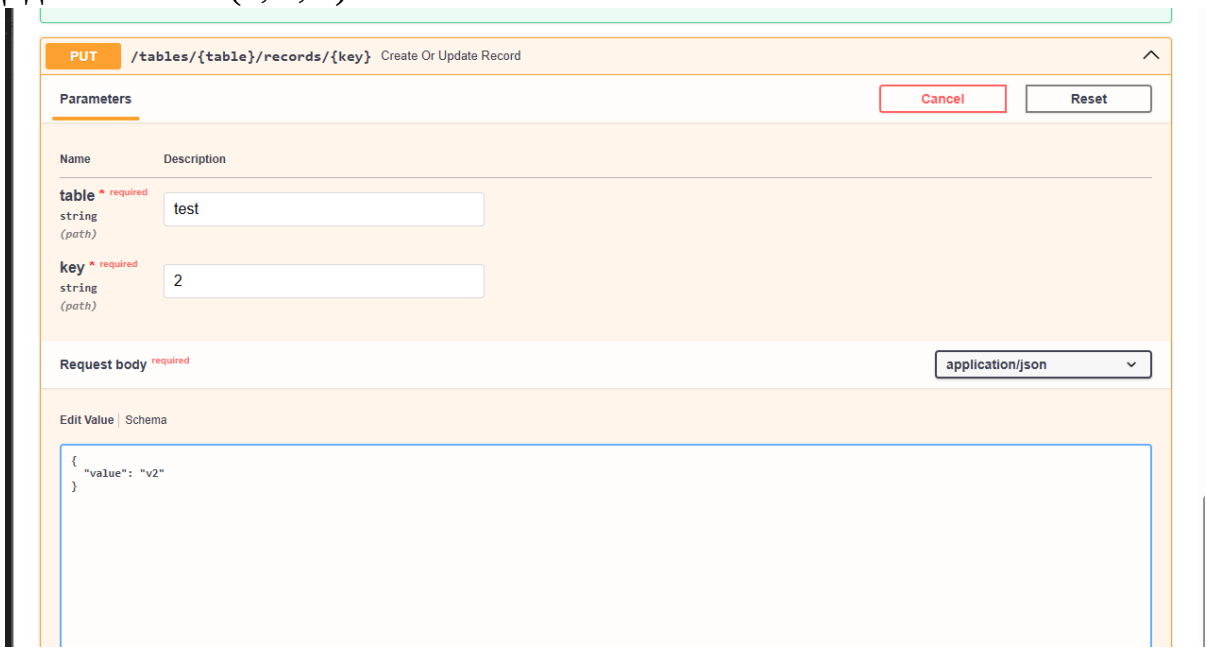
### 5.1 For reads implement quorum with R+W>N rule

Нам треба використати кворумне читання (гарантувати, що прочитане значення є актуальним, навіть якщо частина реплік відстає або недоступна)

У системі з реплікацією для кожного shard-а визначаються параметри:

1. N - кількість реплік shard, N = 3 (leader + 2 followers
2. W- кількість реплік, які мають підтвердити запис W = 1 (запис на leader)
3. R - кількість реплік, з яких coordinator читає дані (R + W> N)

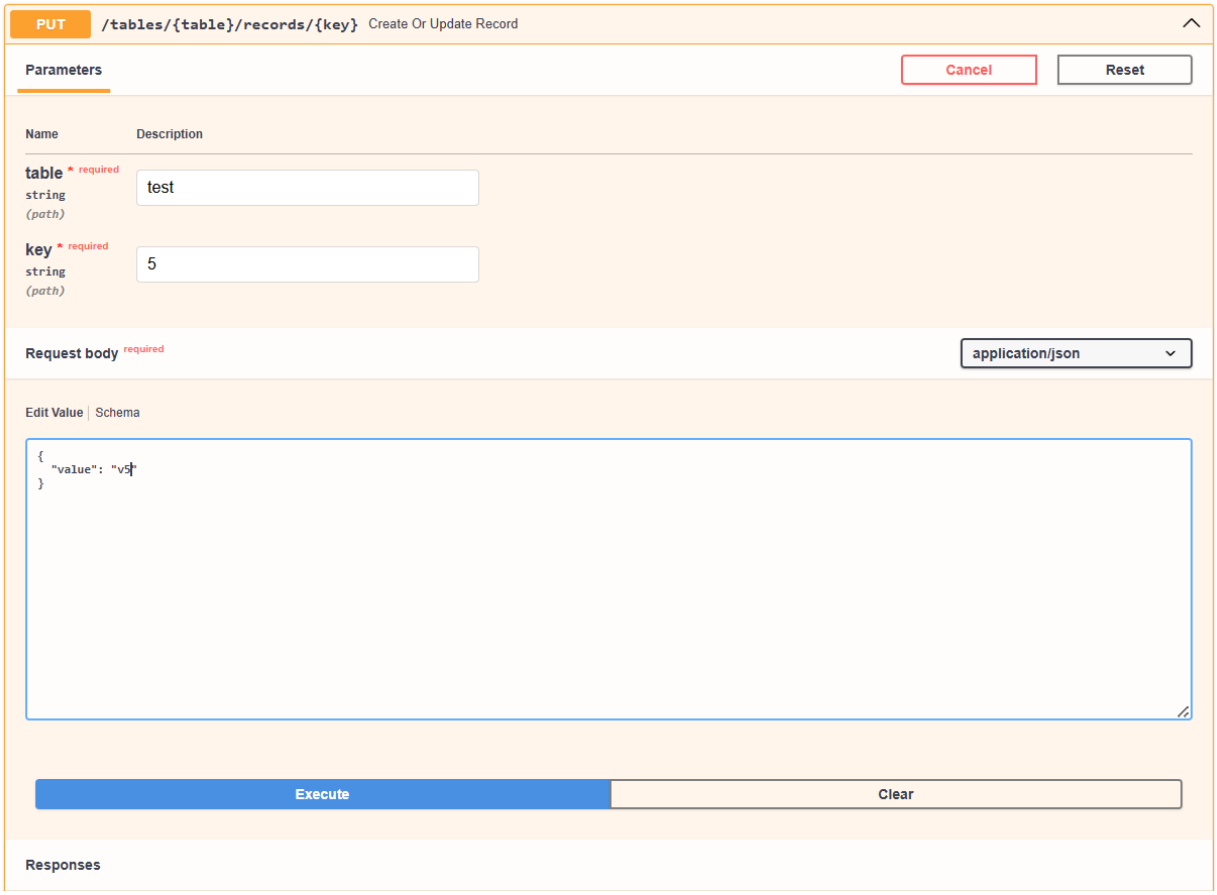
Додамо запити (1, 2, 3)



Зупинемо folower1

```
PS C:\Users\user\PycharmProjects\coordinator> docker stop shard1-follower1
shard1-follower1
```

Зробимо ще кілька запитів:




Маємо такий результат:

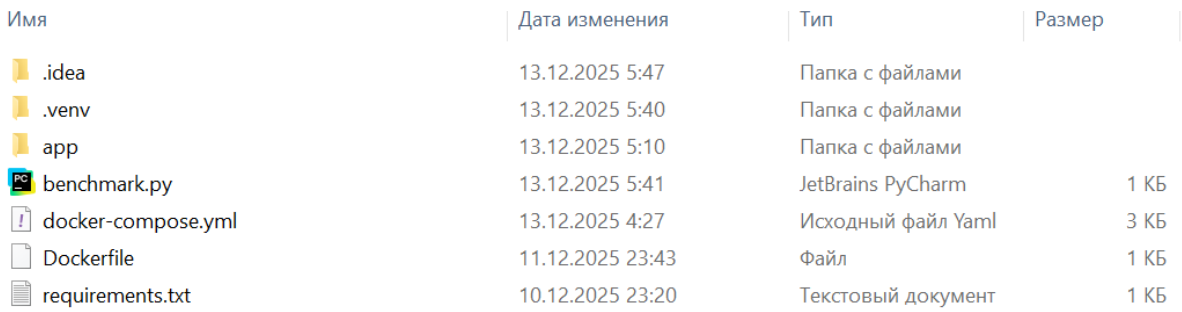



quorum_size у відповіді показує кількість реплік, які реально відповіли на read-запит у конкретний момент. Coordinator завжди намагається звернутися до всіх реплік shard-a, але через перезапуски або недоступність окремих вузлів фактична кількість відповідей може змінюватися (read перетинається з write через leader або актуальну follower-репліку, що відповідає правилу R + W > N.)

## 5.2 Benchmark your solution with YCSB.

Зробимо оцінку продуктивності розробленої розподіленої системи з підтримкою реплікації, quorum reads та балансування навантаження. Для цього необхідно виміряти час виконання операцій запису та читання, а також пропускну здатність системи під паралельним навантаженням. (оскільки розроблена система має власний HTTP API і не підтримується стандартними драйверами YCSB, для тестування було реалізовано YCSB-like benchmark, який імітує типові workload-и YCSB)

Навантажувальне тестування виконувалося за допомогою зовнішнього клієнта benchmark.py, який запускався окремо від сервісів системи та взаємодіяв із нею виключно через HTTP API coordinator.



| Имя | Дата изменения | Тип | Размер |
|---|---|---|---|
| .idea | 13.12.2025 5:47 | Папка с файлами | |
| .venv | 13.12.2025 5:40 | Папка с файлами | |
| app | 13.12.2025 5:10 | Папка с файлами | |
| benchmark.py | 13.12.2025 5:41 | JetBrains PyCharm | 1 КБ |
| docker-compose.yml | 13.12.2025 4:27 | Исходный файл Yaml | 3 КБ |
| Dockerfile | 11.12.2025 23:43 | Файл | 1 КБ |
| requirements.txt | 10.12.2025 23:20 | Текстовый документ | 1 КБ |

```
PS C:\Users\user\PycharmProjects\coordinator> python benchmark.py
PUT time: 0.5745389461517334
GET time: 0.4965171813964844
Throughput ops/sec: 186.73157723099456
```

З результатів видно, що

1. час виконання операцій запису 0.57 с
2. час виконання операцій читання  0.50 с
3. загальна пропускна здатність ~ 186 операцій/сек

Отримані результати відображають сумарний час обробки 200 операцій (100 записів та 100 читань) при паралельному навантаженні.

**Висновок**

У ході виконання лабораторної роботи було розроблено розподілену систему зберігання даних із підтримкою шардингу, реплікації та забезпеченням відмовостійкості. Для кожного shard-а було реалізовано replica set з одним leader та декількома follower-вузлами, що забезпечує збереження даних у разі збоїв окремих компонентів. Запис даних виконується через leader-вузол із подальшою реплікацією на follower-репліки, тоді як операції читання реалізовано з використанням механізму quorum reads, що гарантує отримання актуальних даних навіть за відставання окремих реплік. Також було реалізовано механізм відстеження offset та повторної синхронізації follower-реплік після перезапуску. Для оцінки продуктивності системи проведено навантажувальне тестування за допомогою YCSB-like benchmark, результати якого підтвердили стабільну роботу системи під паралельним навантаженням та коректність реалізованих механізмів масштабування і реплікації.

## Контрольні питання

**Benefits of replicating partitions (shards). Drawbacks as well.**

Реплікація шардів підвищує надійність і доступність системи, оскільки дані зберігаються на кількох вузлах і не втрачаються при відмові одного з них. Також репліки дозволяють розподіляти навантаження на читання, що підвищує масштабованість. Недоліками є збільшене споживання ресурсів, ускладнення архітектури та можливі проблеми з узгодженістю даних між репліками.

**Explain R/W quorum math and why R + W > N yields consistent reads.**

У quorum-моделі N - це загальна кількість реплік, W - кількість реплік, що підтверджують запис, а R - кількість реплік, з яких виконується читання. Якщо R + W > N, то множини реплік для читання і запису обов’язково перетинаються хоча б в одній репліці. Це гарантує, що читання побачить щонайменше одну актуальну версію даних, що забезпечує узгодженість.

**Compare leader-based vs leaderless replication; when would you prefer each?**

У leader-based реплікації всі записи проходять через одного leader-вузла, що спрощує узгодженість і конфлікт-резолюцію, але створює єдину точку запису. Leaderless реплікація дозволяє записувати дані на будь-яку репліку, що підвищує доступність, але потребує складніших механізмів узгодження. Leader-based підходить для систем, де важлива простота і сильна узгодженість, а leaderless - для сценаріїв з високою доступністю і георозподіленими кластерами.

**What’s the role of an append-only event log?**

Append-only event log використовується для впорядкованої фіксації всіх змін стану системи. Він дозволяє відтворювати стан реплік, виконувати реплікацію через fan-out та забезпечує відновлення після збоїв шляхом повторного застосування подій. Такий лог також спрощує контроль порядку операцій і версіювання даних.

**How do write-ack strategies (sync vs async followers) affect tail latency and durability?**

Синхронні підтвердження запису (sync) підвищують надійність, оскільки запис вважається завершеним лише після збереження на кількох репліках, але збільшують затримки (tail latency). Асинхронні підтвердження (async) зменшують затримку записів, проте можуть призвести до втрати останніх змін у разі відмови leader-а до реплікації на followers.

**What failure modes lead to temporary inconsistency? How does your design bound staleness?**

Тимчасова неузгодженість виникає при мережевих затримках, перезапуску реплік або асинхронній реплікації, коли followers відстають від leader-а. У реалізованому дизайні застарілість обмежується використанням версій і quorum reads, оскільки coordinator повертає значення з найбільшою версією, а followers після перезапуску синхронізуються з leader через event log.